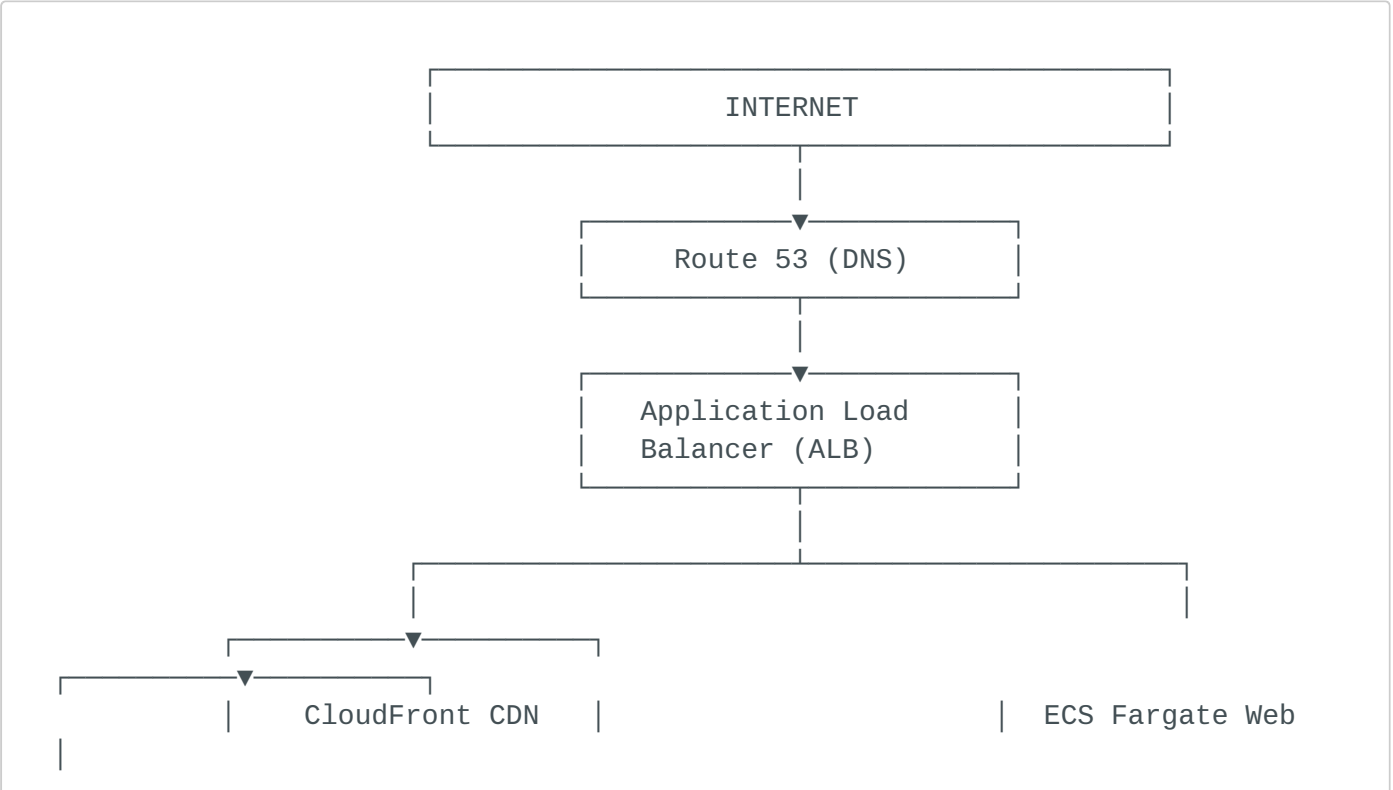
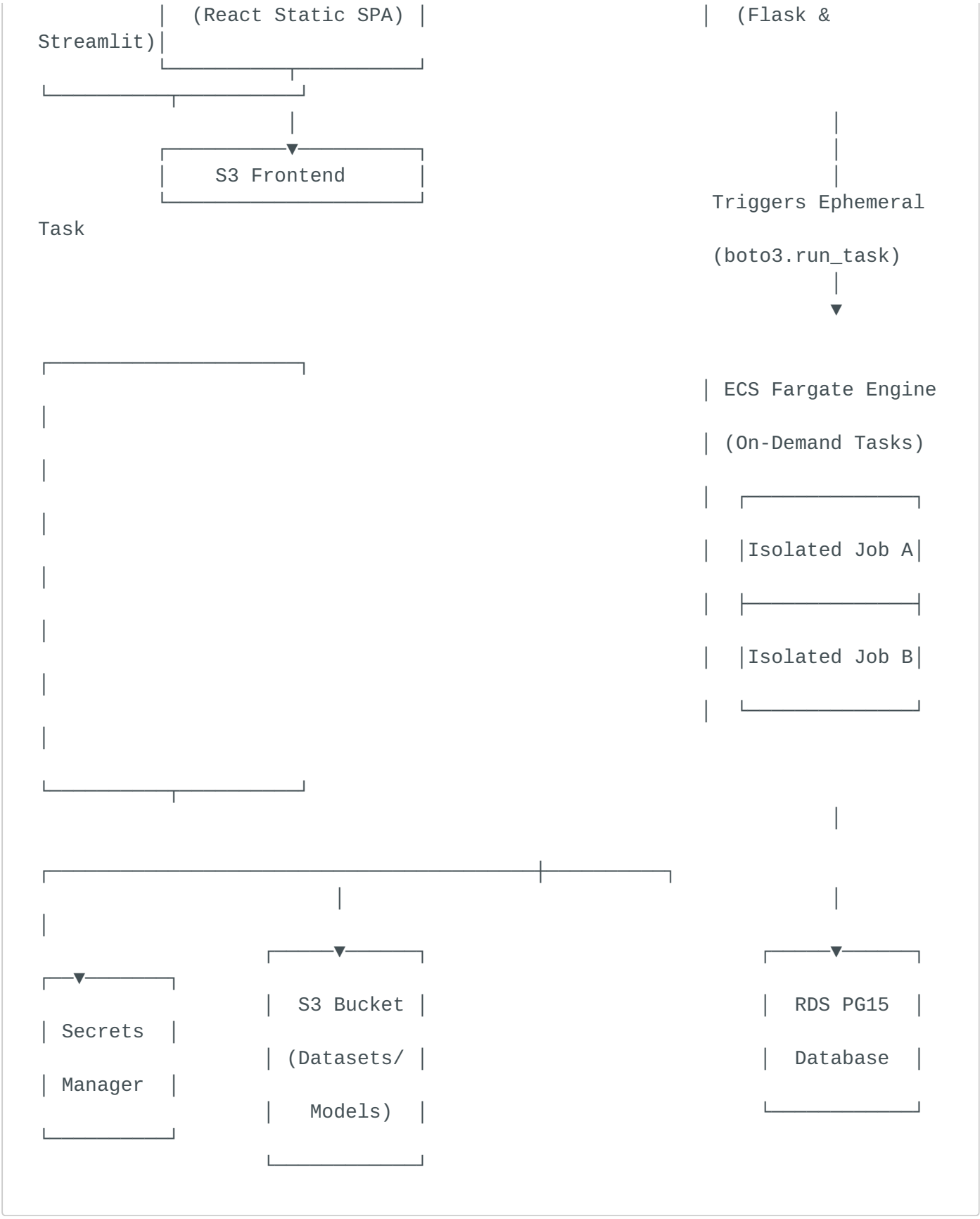
- MLStack — Serverless AWS Ephemeral Task Deployment Guide
  - 🗺️ Serverless Architecture Overview
  - 💎 Why the Ephemeral Task Pattern is Superior
  - 🛠️ Code Implementation
    - 1. The Ephemeral Task Python Launcher
    - 2. The Standalone Training Runner Script
  - 🏗️ AWS Cloud Infrastructure Setup
    - 1. Task Definition Specification
    - 2. IAM Policy Permissions for the Web Task
  - 💰 Cost Analysis (ap-south-1)

MLStack — Serverless AWS Ephemeral Task Deployment Guide

This guide details the **Serverless Ephemeral Task Architecture** for **MLStack**. By leveraging on-demand isolated CPU containers instead of a persistent Celery worker pool, you guarantee **absolute resource isolation, infinite scaling, and zero idle server costs**.

🗺️ Serverless Architecture Overview





💎 Why the Ephemeral Task Pattern is Superior

1. **Absolute Resource Isolation**: Every single training job runs in a completely separate, dedicated Fargate micro-virtual machine with its own defined vCPU and RAM. A heavy XGBoost run by User A can never slow down or crash User B's training run.
2. **Infinite Scaling**: If 100 users click "Train" at the exact same moment, AWS instantly spins up 100 independent CPU Fargate tasks. They all execute in parallel without any queue blocking.
3. **Zero Idle Compute Cost**: There is no persistent Celery worker container running and incurring costs 24/7. When there are no training runs in progress, your training compute cost is **exactly $0.00**.
4. **Simplified Infrastructure**: You no longer need to manage a Celery scheduler, a Celery worker pool, or scale-out auto-scaling groups. AWS handles the entire scaling lifecycle on a per-task basis.

🛠️ Code Implementation

To execute this architecture, we replace Celery dispatch in the Flask backend with a call to the AWS ECS SDK (`boto3`).

1. The Ephemeral Task Python Launcher

Create a new utility `backend/app/services/ecs_service.py` to trigger the standalone container:

```
# backend/app/services/ecs_service.py
import os
import boto3
from flask import current_app

def launch_training_task_on_fargate(experiment_id: int):
    """
    Launches an isolated, ephemeral CPU container on AWS Fargate
    to execute a single AutoML training job.
    """
    ecs_client = boto3.client('ecs', region_name=os.getenv('AWS_REGION',
'ap-south-1'))

    cluster_name = os.getenv('ECS_CLUSTER_NAME', 'mlstack-prod')
    task_definition = os.getenv('ECS_TASK_DEFINITION', 'mlstack-training-
task:latest')
    subnet_ids = os.getenv('ECS_SUBNET_IDS', '').split(',')
```

```
    security_group_ids = os.getenv('ECS_SECURITY_GROUPS', '').split(',')

    # Inject Experiment ID into the standalone execution container
    container_override = {
        'name': 'mlstack-training-runner',
        'command': ['python', 'standalone_runner.py', '--experiment_id',
str(experiment_id)],
        'environment': [
            {'name': 'EXPERIMENT_ID', 'value': str(experiment_id)},
            {'name': 'FLASK_ENV', 'value': 'production'}
        ]
    }

    response = ecs_client.run_task(
        cluster=cluster_name,
        taskDefinition=task_definition,
        launchType='FARGATE',
        networkConfiguration={
            'awsvpcConfiguration': {
                'subnets': subnet_ids,
                'securityGroups': security_group_ids,
                'assignPublicIp': 'DISABLED'  # Runs fully isolated in
private subnets
            }
        },
        overrides={
            'containerOverrides': [container_override]
        }
    )

    task_arn = response['tasks'][0]['taskArn']
    current_app.logger.info(f"Successfully launched isolated training task
{task_arn} for Experiment {experiment_id}")
    return task_arn
```

---

2. The Standalone Training Runner Script

Create the execution target script `/home/bharatkumar/Documents/projects/MLStack/backend/standalone_runner.py` inside your backend folder. This script is executed by the Fargate container on startup, performs the training, and then terminates.

```
# backend/standalone_runner.py
import argparse
import sys
import os

# Add root directory to python path
sys.path.append(os.path.dirname(os.path.abspath(__file__)))
```

```
from app import create_app, db
from app.models.experiment import Experiment
from app.tasks.training_tasks import run_training_pipeline
from app.services.minio_service import get_minio_service
from app.models.dataset import Dataset
from datetime import datetime
import pandas as pd
import tempfile

def run_isolated_training(experiment_id: int):
    app = create_app()
    with app.app_context():
        experiment = db.session.get(Experiment, experiment_id)
        if not experiment:
            print(f"Error: Experiment {experiment_id} not found.")
            sys.exit(1)

        try:
            print(f"Starting isolated training run for Experiment:
{experiment.name} (ID: {experiment_id})")

            # Update status in PostgreSQL database
            experiment.status = 'training'
            db.session.commit()

            # Fetch dataset info
            dataset = db.session.get(Dataset, experiment.dataset_id)
            if not dataset:
                raise ValueError("Dataset not found")

            # Download dataset from S3/MinIO
            minio = get_minio_service()
            with tempfile.NamedTemporaryFile(suffix='.csv', delete=False) as
tmp:
                tmp_path = tmp.name

            minio.download_file(minio.BUCKET_DATASETS, dataset.file_path,
tmp_path)

            # Load into pandas memory
            df = pd.read_csv(tmp_path)
            os.unlink(tmp_path)  # Clean up temp file

            # Execute core training pipeline
            result = run_training_pipeline(experiment=experiment, df=df,
task=None)

            # Save successful outputs
            experiment.status = 'completed'
            experiment.completed_at = datetime.utcnow()
            experiment.best_model_name = result.get('best_model')
            experiment.best_score = result.get('best_score')
            experiment.best_model_id = result.get('model_path')
            db.session.commit()

            print(f"✓ Isolated training completed successfully. Best Model:
```

```
  {experiment.best_model_name} (Score: {experiment.best_score})")

        except Exception as e:
            print(f"❌ Isolated training failed: {str(e)}")
            experiment.status = 'failed'
            db.session.commit()
            sys.exit(1)

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Isolated MLStack Training
Runner")
    parser.add_argument("--experiment_id", type=int, required=True,
help="Experiment ID to train")
    args = parser.parse_args()

    run_isolated_training(args.experiment_id)
```

🏗️ AWS Cloud Infrastructure Setup

1. Task Definition Specification

You create **two separate ECS Fargate task definitions**:

1. `mlstack-web-task`: Runs the 24/7 web services (Flask backend, Streamlit predictor, Node mailer).
2. `mlstack-training-task` (Ephemeral): Created specifically for training. Only runs on demand.

`mlstack-training-task` **Fargate Specs:**

- **vCPU**: `2 vCPU` (Standard multicore for Scikit-Learn/XGBoost training)
- **Memory**: `4 GB` (Sufficient for loading tabular datasets into Pandas)
- **Compatibility**: Fargate (CPU-only, low cost)
- **Container Image**: Points to your primary `mlstack/backend` ECR image

---

2. IAM Policy Permissions for the Web Task

For your Flask API server to trigger these Fargate tasks, the IAM Role attached to the **Web task** must contain the following policy:

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Sid": "ECSTaskExecutionAccess",
            "Effect": "Allow",
            "Action": [
                "ecs:RunTask"
            ],
            "Resource": [
                "arn:aws:ecs:ap-south-1:<ACCOUNT_ID>:task-
definition/mlstack-training-task:*"
            ]
        },
        {
            "Sid": "PassRoleAccess",
            "Effect": "Allow",
            "Action": [
                "iam:PassRole"
            ],
            "Resource": [
                "arn:aws:iam::<ACCOUNT_ID>:role/ecsTaskExecutionRole",
                "arn:aws:iam::<ACCOUNT_ID>:role/mlstackTaskRole"
            ]
        }
    ]
}
```

💰 Cost Analysis (ap-south-1)

With this ephemeral setup, you only pay for compute *while* models are actively training:

- **Fargate CPU Pricing**: ~$0.04048 per vCPU-hour, $0.004445 per GB-hour.
- **A Standard Training Job**: Runs on 2 vCPUs and 4 GB RAM for 5 minutes (0.083 hours):

$$\text{Cost} = (2 \times 0.04048 + 4 \times 0.004445) \times 0.083 \approx \$0.008 \text{ per training job}$$

This means **you can run over 120 isolated training pipelines for less than $1.00 USD!** When your app is idle at night, your training compute cost drops to absolute zero.